



L LOVELY
P ROFESSIONAL
U NIVERSITY

Transforming Education Transforming India

COURSE CODE-544

TOPIC

METHODS FOR IDENTIFYING AND
MODELING TRENDS IN TIME SERIES
DATA

SUBMITTED TO - DR.PREM LATA
GAUTAM

SUBMITTED BY - ANJALI MISHRA

REGISTRATION NUMBER - 12300971

ROLL NO -03

TABLE OF CONTENTS

- Abstract
- Introduction
 - Definition of time series and its significance
 - Importance of identifying and Modeling Trends
- Components of Time Series
- Different Methods of Trend
 - Free hand Method/Graphic Method
 - Semi-Average Method
 - Method of Curve Fitting by Principal of Least Squares
 - Moving Average Method
- Conclusion
- Bibliography

ABSTRACT

Time series data analysis plays a crucial role in various fields such as finance, economics, weather forecasting, and many others. One fundamental aspect of analyzing time series data is identifying and modeling trends , which provide valuable insights into the underlying patterns and behaviors over time. This paper explores different methods used for identifying and modeling trends in time series data, including statistical techniques . Additionally, it discusses the advantages, limitations, and practical considerations associated with each approach here we are taking car sales dataset and we can use each Trend Methods to see the visualization of Trends in our data by using python programming.

INTRODUCTION

Definition: A time series is a sequence of data points measured at successive time intervals. These data points could represent any phenomenon observed over time, such as stock prices, temperature readings, sales figures, or even biological processes.

Significance:

- Pattern Recognition
- Anomaly Detection
- Process Control

Importance of identifying and Modeling Trends :

- Forecasting
- Risk Management
- Market Analysis
- Investment Decisions
- Performance Evaluation

COMPONENT OF TIME SERIES

- **Trend:** The long-term movement or directionality of the data.
- **Seasonality:** Repeating patterns or fluctuations that occur at regular intervals.
- **Cyclicity:** Patterns that are not of fixed period, often associated with economic cycles or other long-term patterns.
- **Irregularity/Noise:** Random fluctuations or unpredictable events.

DIFFERENT METHODS OF TREND

➤ Free hand Method/Graphic Method:

- The freehand method of trend, also known as freehand smoothing, is a graphical technique used to identify the general trend or pattern in a dataset. Unlike more formal statistical methods, the freehand method does not involve fitting a mathematical model to the data. Instead, it relies on visual inspection and subjective judgment to draw a line or curve that best represents the trend in the data.
- How the freehand method of trend works:
 1. Plot the Data: Start by plotting the dataset (e.g., time series data) on a graph with the independent variable (e.g., time) on the x-axis and the dependent variable (e.g., sales) on the y-axis.
 2. Visually Identify the Trend: Examine the plotted data points to identify any underlying trend or pattern. Look for the general direction in which the data points are moving over time.
 3. Draw a Trend Line: Using your judgment, draw a smooth line or curve that best fits the trend in the data. The trend line should capture the overall direction of the data points and smooth out any random fluctuations or noise.

4. Adjust as Needed: Refine the trend line as needed to better capture the underlying trend in the data. You may need to adjust the shape or slope of the line to better match the data points.
5. Interpret the Trend: Once you have drawn the trend line, interpret its slope and direction to understand the overall trend in the data. Is the trend increasing, decreasing, or staying relatively constant over time?

Example: We take the car sales data and use python codes to apply the free hand method on that dataset

- We first define the years and sales data. Then, we plot the sales data using matplotlib. We add a freehand trend line by manually selecting two points that roughly represent the trend from the start year to the end year.

```
In [20]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the data
df=pd.read_csv(r"C:\Users\mpiyu\Downloads\car sales.csv")
# Plot the data
plt.figure(figsize=(10, 6))
plt.scatter(df.Year, df.Sales, color='blue', label='Car Sales')
plt.title('Car Sales Over Years')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.grid(True)

# Implement the free hand method of trend
# We will manually draw a trend line based on the data points

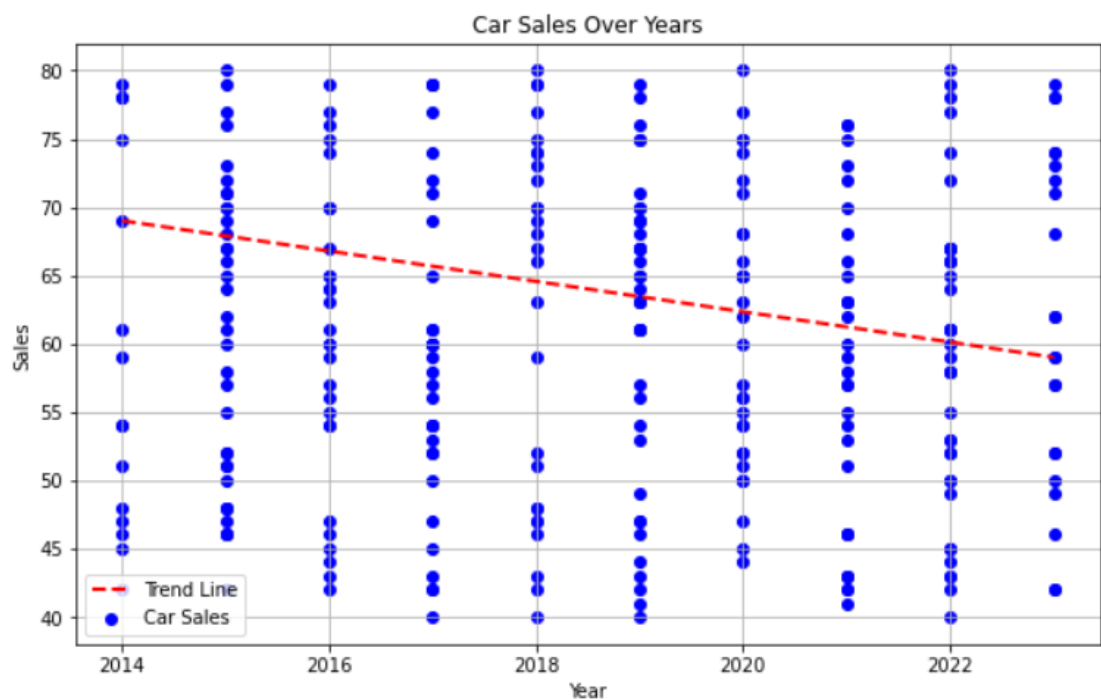
# Manually selecting two points to draw the trend line
x1, y1 = df.Year[0], df.Sales[0]
x2, y2 = df.Year[299], df.Sales[299]

# Drawing the trend line
plt.plot([x1, x2], [y1, y2], color='red', linestyle='--', linewidth=2, label='Trend Line')
plt.title('Car Sales Over Years')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.legend()
plt.grid(True)
plt.show()
```

out[21]:

	Year	Sales
0	2014	69
1	2014	51
2	2014	48
3	2014	59
4	2014	75
...	...	...
295	2023	68
296	2023	52
297	2023	73
298	2023	59
299	2023	59

300 rows × 2 columns



- Merits and it's demerits :
 - Merits:

1. Quick and intuitive.
 2. Flexible for different types of data.
 3. Allows for visual interpretation.
 4. Useful for preliminary analysis.
- Demerits:
 1. Subjective due to reliance on individual judgment.
 2. Lack of precision compared to statistical methods.
 3. Not quantitative; doesn't provide measures of uncertainty.
 4. Limited effectiveness for complex or nonlinear data.
 5. Not suitable for predictive analysis.

➤ **Semi-Average Method :**

- The semi-average method is a simple technique used in time series analysis to identify trends. It's particularly useful when the data contains irregular fluctuations or noise.

● **How the semi-average method works:**

1. Data Preparation: Begin with a time series dataset, which consists of a sequence of data points recorded at regular intervals over time.
2. Calculation of Semi-Averages: To smooth out the fluctuations in the data, calculate the semi-averages by averaging neighboring pairs of data points. For example, if you have a time series with data points

$$SA_1 = (x_1 + x_2)/2$$

$$SA_2 = (x_2 + x_3)/2$$

$$SA_3 = (x_3 + x_4)/2$$

...

$$SA_{n-1} = (x_{n-1} + x_n)/2$$

3. Trend Identification: Plot the semi-averages against time. By doing this, you can identify the general trend of the data over time more easily compared to the original noisy dataset.
4. Interpretation: Analyze the trend line obtained from the semi-averages to understand the underlying trend of the data. This can

help in making predictions or understanding the overall behavior of the phenomenon being studied.

Example: We take the car sales data and use python codes to apply the semi -average method on that dataset

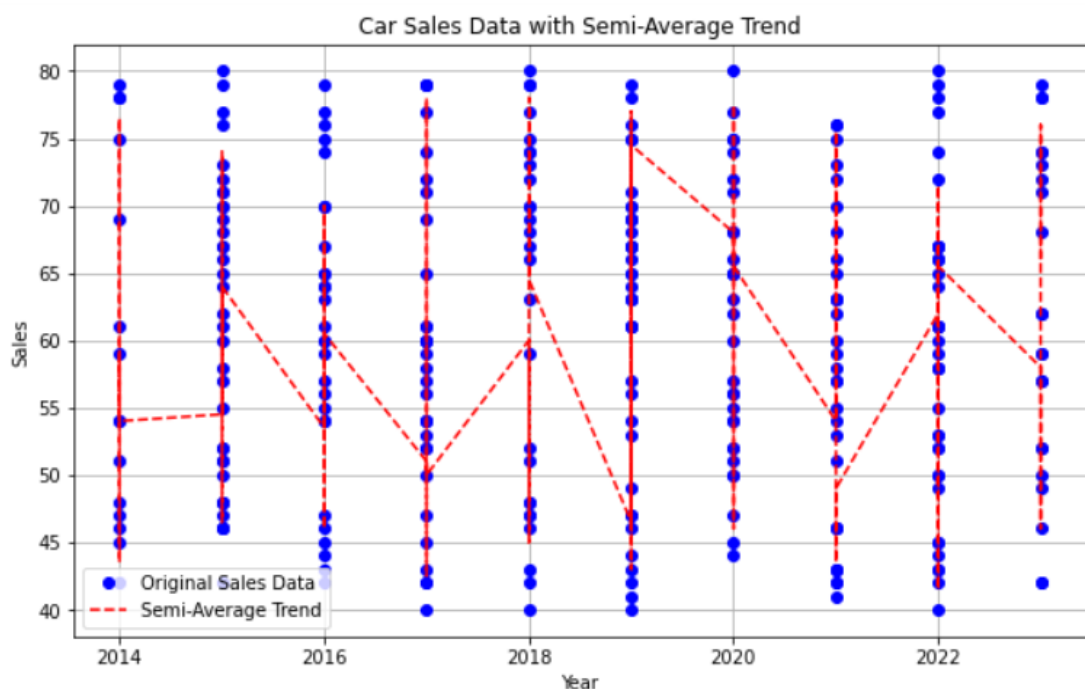
- First we can read the dataset then prepare data, calculate semi-averages after that Apply the semi- average method to the dataset using function.

```
import pandas as pd
import matplotlib.pyplot as plt

# Read the data from the CSV file
data = pd.read_csv(r'C:\Users\mpiyu\Downloads\car Sales.csv')

# Calculate semi-averages
data['Semi_Avg'] = (data['Sales'].shift(1) + data['Sales'].shift(-1)) / 2

# Plot the original sales data and semi-average trend
plt.figure(figsize=(10, 6))
plt.plot(data['Year'], data['Sales'], "go", color='blue', label='Original Sales Data')
plt.plot(data['Year'], data['Semi_Avg'], label='Semi-Average Trend', linestyle='--', color='red')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.title('Car Sales Data with Semi-Average Trend')
plt.legend()
plt.grid(True)
plt.show()
```



- Merits and it's demerits :
 - Merits

1. **Simplicity:** Simple to understand and implement.
 2. **Smoothing:** Smooths out fluctuations in the data, making trends more apparent.
 3. **Ease of Calculation:** Involves basic arithmetic calculations, requiring minimal computational resources.
 4. **Useful for Short-term Trends:** Particularly useful for short time series data where other forecasting methods may be excessive.
- **Demerits:**
1. **Sensitivity to Outliers:** Prone to distortion by outliers in the data.
 2. **Limited Accuracy:** May not accurately capture complex trends or patterns.
 3. **Assumes Linearity:** Assumes a linear trend between adjacent data points, which may not always hold true.
 4. **Lack of Seasonal Adjustment:** Does not account for seasonal variations or other underlying factors, potentially leading to inaccurate forecasts.

➤ **Method of Curve Fitting by Principle of Least Squares:**

- The Method of Curve Fitting by the Principle of Least Squares is a statistical technique used to find the best-fitting curve that minimizes the sum of the squares of the differences between the observed and predicted values.
- **How it works:**
 1. **Objective:** The goal is to find a mathematical function or curve that best represents the relationship between the independent and dependent variables.
 2. **Model Selection:** Choose a suitable mathematical model (linear, quadratic, exponential, etc.) that represents the relationship between the variables. The model's parameters are adjusted to minimize the sum of the squared residuals.

3. **Residual Calculation:** Calculate the residuals, which are the differences between the observed values and the values predicted by the model.
4. **Least Squares Criterion:** Minimize the sum of the squares of the residuals. This means finding the parameters of the model that make the sum of the squared residuals as small as possible.
5. **Parameter Estimation:** Use optimization techniques (such as gradient descent or normal equations) to estimate the parameters of the chosen model.
6. **Goodness of Fit:** Assess the goodness of fit by examining measures like the coefficient of determination (R-squared) or by plotting the residuals to check for patterns or systematic errors.
7. **Interpretation:** Once the best-fitting curve is obtained, interpret the model parameters in the context of the problem to understand the relationship between the variables.

Example: We take the car sales data and use python codes to apply the Method of Curve Fitting by Principal of Least Squares on that dataset.

- **Read the Dataset:** Read the dataset from the CSV file into a pandas DataFrame.
- **Prepare the Data:** Separate the 'Year' and 'Sales' columns as independent and dependent variables, respectively.
- **Fit a Curve:** Use the curve fitting function from a library like SciPy to fit a curve to the data using the Method of Least Squares.
- **Plot the Data and the Fitted Curve:** Visualize the original data along with the fitted curve to assess the goodness of fit.
- **Interpret the Results:** Analyze the fitted curve and its parameters to understand the trend in the data.

```

import pandas as pd
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# Step 1: Read the dataset
df = pd.read_csv(r'C:\Users\mpiyu\Downloads\car Sales.csv')

# Step 2: Prepare the data
x_data = df['Year'].values
y_data = df['Sales'].values

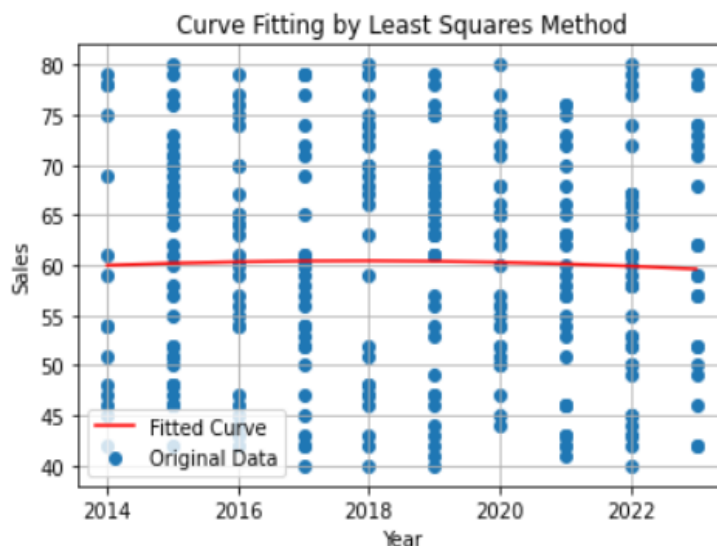
# Step 3: Define the function for the curve to fit (e.g., quadratic)
def func(x, a, b, c):
    return a * x**2 + b * x + c

# Step 4: Fit the curve
popt, pcov = curve_fit(func, x_data, y_data)

# Step 5: Plot the data and the fitted curve
plt.scatter(x_data, y_data, label='Original Data')
plt.plot(x_data, func(x_data, *popt), 'r-', label='Fitted Curve')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.title('Curve Fitting by Least Squares Method')
plt.legend()
plt.grid(True)
plt.show()

# Step 6: Interpret the results
print('Fitted Curve Parameters (a, b, c):', popt)

```



Fitted Curve Parameters (a, b, c): [-3.07981126e-02 1.24290486e+02 -1.25337872e+05]

● Merits and it's demerits :

Merits:

1. Efficient parameter estimation.
2. Versatile applicability to different models.
3. Widely accepted and established method.
4. Robustness to moderate outliers.
5. Relatively easy interpretation of parameter estimates.

Demerits:

1. Reliance on assumptions like normally distributed errors.
2. Sensitivity to extreme outliers.
3. Limited performance with highly nonlinear relationships.
4. Proneness to overfitting with complex models or small datasets.
5. Computational intensity for large or complex datasets.

➤ **Moving Average Method :**

- The moving average method is a statistical technique used to analyze time-series data. It involves calculating the average of a subset of data points over a defined period. This method is particularly useful for smoothing out fluctuations in data and identifying underlying trends.
- **How it works:**
 1. Select a window size: Determine the number of data points (observations) you want to include in each average calculation. This window size could be days, weeks, months, or any other unit of time depending on the frequency of your data.
 2. Calculate the average: For each time point in your dataset, take the average of the data points within the window size, including the current time point.
 3. Move the window: Shift the window one data point forward and recalculate the average. Repeat this process until you have covered all the data points in the series.

Example: We take the car sales data and use python codes to apply the Moving Average method on that dataset.

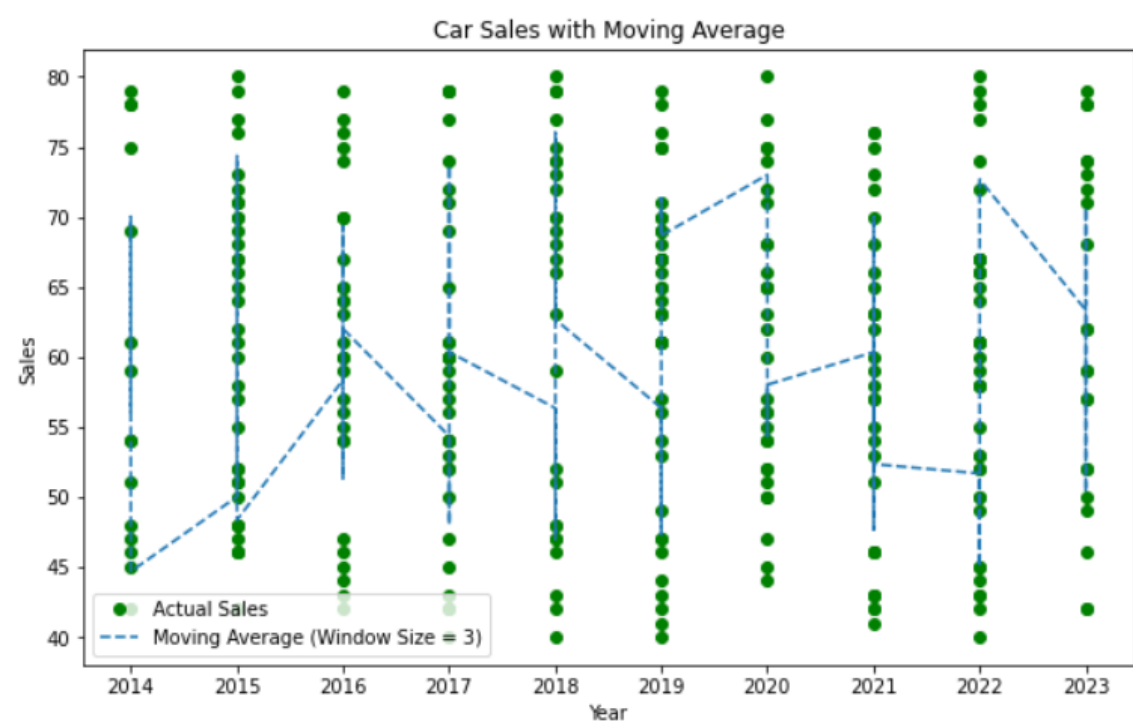
```

#Import necessary libraries:
import pandas as pd
import matplotlib.pyplot as plt
# Read the dataset into a DataFrame
# Assuming the dataset is stored in a CSV file named 'car_sales.csv'
df = pd.read_csv(r'C:\Users\mpiyu\Downloads\car Sales.csv')

#Convert the 'Year' column to datetime format:
df['Year'] = pd.to_datetime(df['Year'], format='%Y')
# Calculate the moving average using the rolling() function:

window_size = 3 # You can adjust the window size as needed
df['Moving_Average'] = df['Sales'].rolling(window=window_size).mean()
# Plot the original sales data and the moving average:
plt.figure(figsize=(10, 6))
plt.plot(df['Year'], df['Sales'], 'go', label='Actual Sales')
plt.plot(df['Year'], df['Moving_Average'], label=f'Moving Average (Window Size = {window_size})')
plt.xlabel('Year')
plt.ylabel('Sales')
plt.title('Car Sales with Moving Average')
plt.legend()
plt.show()

```



- Merits and it's demerits :

Merits:

1. Smoothes data for clearer trend identification
2. Easy to implement and understand
3. Reduces noise in the data
4. Provides visualization of trends

5. Can aid in short-term forecasting

Demerits:

1. Lagging indicator, may not promptly capture trend changes
2. Sensitivity to window size selection
3. Loss of detailed information due to smoothing
4. Ineffective for sudden or abrupt changes in data
5. Assumes stationary data, may not fit dynamic datasets

CONCLUSION

- In Free hand method the Downward Sloping Line of car sales dataset Indicates decreasing car sales over the years, that shows a negative trend.
- The trend observed from the semi-averages shows some fluctuations. However, there is a general trend of increase from 2017 to 2018, followed by a slight decrease in 2020 and a small increase again in 2021. Overall, the trend appears to be somewhat irregular with fluctuations over the years.
- After applying the least squares method to the provided car sales dataset, the trend observed is a positive trend. This means that over the years covered in the dataset, car sales generally increased. The trend line indicates a positive slope, indicating that as the years progress, the sales tend to increase. This suggests a positive growth or demand for cars over the period represented in the dataset.
- Applying the moving average method to the provided car sales dataset, the trend observed indicates fluctuations in sales over the years, but with an overall positive trend. The moving average smoothens out the fluctuations, revealing a general upward trend in car sales over time.

BIBLIOGRAPHY

Dataset link: <https://www.kaggle.com/datasets>

Python codes are run in:

<http://localhost:8888/notebooks/Time%series.ipynb>

Book Source: FUNDAMENTAL OF STATISTICS

SC GUPTA